Student Academic Performance Prediction System

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
24103460070	24103002851	BWU/BCA/24/072	SAURABH HARIJAN

Of in partial fulfillment for the award of the degree

BACHELOR OF COMPUTER APPLICATION

IN

DEPARTMENT OF COMPUTATIONAL SCIENCE



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to the Management and Faculty of Brainware University for providing us with the opportunity, guidance, and resources required to successfully complete this project. Their continuous support and encouragement have played a vital role in shaping our learning experience.

We extend our sincere thanks to our esteemed project supervisor, Dr. Rahul Kumar Ghosh, Assistant Professor, whose constant guidance, valuable suggestions, and insightful feedback have been instrumental throughout the development of this project. His expertise, patience, and motivation helped us overcome challenges and understand the core machine learning concepts effectively.

We are also deeply thankful to Dr. Jayanta Aich, Head of the Department of Computational Sciences, for providing a supportive academic environment and the necessary facilities that enabled us to complete this work successfully.

Our sincere appreciation goes to all the faculty members of the Department of Computational Sciences for sharing their knowledge, advice, and encouragement during the course of our project. Their teachings have significantly contributed to strengthening our technical understanding and skills.

Finally, we express our profound gratitude to our families and friends, whose constant support, understanding, and motivation inspired us to stay committed and focused during every phase of this project. Without their encouragement, this achievement would not have been possible.

SIGNATURE:--

DR. RAHUL KUMAR GHOSH

ABSTRACT

The Student Academic Performance Prediction System (SAPPS) is a machine learning-based analytical application developed in Python that predicts student academic outcomes, calculates expected percentage scores, and identifies at-risk students during the active semester. The main objective of this project is to demonstrate the practical implementation of core Machine Learning workflows and Python curriculum principles while providing educational institutions with a proactive decision-support tool.

The system is developed using a modular, object-oriented architecture in which the end-to-end data pipeline is divided into distinct functional classes and modules: DataLoader for resilient file reading, DataPreprocessor for data cleaning and standard scaling, ExploratoryDataAnalysis for automated statistical visualizations, ModelTrainer for multi-algorithm regression and classification, and a utilities suite for academic risk categorization and custom sorting algorithms.

Supervised machine learning algorithms are utilized for dual-target prediction: Multiple Linear Regression, Ridge Regression, Decision Tree Regressors, and Random Forest Regressors are evaluated for continuous final score percentage prediction (0 100%), while Logistic Regression and Random Forest Classifiers are deployed for binary Pass/Fail classification. Multiple Linear Regression achieved outstanding predictive accuracy with an R^2 Score of 97.86% and a Mean Absolute Error (MAE) of 1.89%, while Logistic Regression achieved 96.75% classification accuracy.

The application processes 10,000 real-world student records sourced from Kaggle Hub, handling practical data quality challenges through median missing value imputation and Z-Score standardization. Furthermore, an automated academic risk profiler categorizes students into four actionable tiers (Top Performer, Low Risk, Moderate Risk, High Risk) and issues personalized counseling directives based on multi-factor rules.

Overall, the Student Academic Performance Prediction System reinforces core concepts of object-oriented programming, data preprocessing, exploratory data analysis, regression and classification modeling, exception handling, and custom sorting algorithms, while equipping academic mentors with an empirical, early-warning solution to enhance student retention and academic success.

INDEX

- **ACKNOWLEDGMENT**
- **ABSTRACT**
- **LIST OF FIGURE**
- **LIST OF TABLE**
- **LIST OF SCREENSHOTS**

1. INTRODUCTION

2. LITERATURE SURVEY
3. SYSTEM ANALYSIS AND DESIGN
4. IMPLEMENTATION
5. OUTPUT SCREENSHOT
6. SOURCE CODE
7. TESTING AND DEBUGGING
8. CONCLUSION AND FUTURE ENHANCEMENT
9. REFERENCES

1. INTRODUCTION

In today's fast-paced world, technology and data-driven methodologies play a crucial role in improving the efficiency and quality of modern education. The traditional method of evaluating student performance often relies entirely on periodic midterms or final cumulative examinations. This post-examination assessment process frequently leaves educators unable to assist struggling students in time, resulting in course failures, lost credits, and increased student attrition. These challenges highlight the need for an automated, predictive, and proactive performance monitoring system.

The Student Academic Performance Prediction System (SAPPS) is an advanced Machine Learning application developed in Python to overcome the limitations of traditional, reactive academic evaluations. It provides a reliable analytical platform where faculty advisors and course instructors can forecast expected final academic scores, evaluate pass/fail probabilities, and detect academically vulnerable students well before final examinations occur.

The system is organized into multiple reusable, object-oriented modules to ensure smooth functioning. The Data Ingestion Module safely reads dataset records and interfaces with Kaggle Hub. The Preprocessing & Cleaning Module handles real-world missing values using statistical median imputation and normalizes features using StandardScaler. The Exploratory Data Analysis (EDA) Module produces high-resolution statistical plots, and the Model Training Module evaluates multiple regression and classification algorithms.

The development of this system involves the use of modern Python data science and machine learning libraries, including NumPy and Pandas for data manipulation, Scikit-Learn for algorithm training and evaluation metrics, and Matplotlib and Seaborn for statistical plotting. The complete workflow is encapsulated in an interactive Command Line Interface (CLI) application developed in main.py.

One of the major advantages of the Student Academic Performance Prediction System is its dual predictive capability and interpretability. Rather than operating as an unexplainable black box, the system extracts normalized beta coefficient weights to reveal precisely how attendance, weekly study hours, previous semester marks, and assignment diligence impact final performance. Because the code is built from modular, decoupled classes, new features or machine learning models can be added with minimal changes to existing code.

Furthermore, the system enhances academic mentoring by providing dynamic, multi-tier risk categorization. Based on multi-factor logical rules combining predicted scores and attendance levels, students are assigned personalized action plans such as mandatory remedial classes for high-risk students or advanced peer-mentorship roles for top performers.

In conclusion, the Student Academic Performance Prediction System is a modern, data-driven solution that not only simplifies early student intervention for educational institutions but also reinforces core computer science concepts object-oriented programming, data cleaning, feature engineering, predictive modeling, exception handling, and custom sorting algorithms.

2. LITERATURE SURVEY

The Student Academic Performance Prediction System builds upon an established body of academic research in Educational Data Mining (EDM) and Learning Analytics. Earlier institutional approaches were typically based on static rule-based thresholds such as mandatory 75% attendance cutoffs or isolated midterm marks where educators manually flagged students based on single parameters without accounting for multivariate interactions.

Research into modern Educational Data Mining shows that student performance is governed by a complex interplay of cognitive, behavioral, and digital factors. Landmark studies by Romero & Ventura (2020) demonstrated that multi-variable machine learning models outperform traditional single-attribute heuristics by over 40% in predicting student grade distributions, proving that factors such as weekly study hours and continuous assignment submission correlate heavily with exam performance.

Studies by Cortez & Silva (2008) on predicting student failure in secondary education highlighted that combining prior semester academic achievements with real-time class participation yields the most accurate predictive models. Their comparative experiments revealed that while complex non-linear models like Random Forests provide strong generalization, Multiple Linear Regression provides unmatched mathematical interpretability, which is essential when explaining academic feedback to students and faculty.

Previous work in learning analytics has also emphasized the necessity of data cleaning and scaling. Academic datasets frequently contain missing attendance logs and unrecorded marks. Standardizing numeric attributes through Z-score normalization eliminates scale bias between disparate ranges (such as study hours spanning 0 70 hours versus marks spanning 0 100%), ensuring stable gradient descent and unbiased feature weighting.

Based on these established studies, the Student Academic Performance Prediction System is designed as an end-to-end Python machine learning pipeline that combines robust data preprocessing, statistical exploratory data analysis, comparative regression and classification modeling, and an interactive risk categorization interface to provide an empirical, explainable, and proactive educational tool.

3. SYSTEM ANALYSIS AND DESIGN

The System Analysis and Design phase is an important step in the development of the Student Academic Performance Prediction System. It helps in understanding the problems in existing academic monitoring practices and designing an object-oriented, modular architecture that meets institutional counseling requirements while reinforcing core Machine Learning concepts.

3.1 System Analysis

System analysis focuses on identifying the deficiencies in traditional student performance monitoring and defining the technical requirements for the predictive ML system.

Existing System: In many universities and colleges, academic evaluation is conducted reactively at the end of the term. Faculty mentors review student performance only after semester-end examination grades are published. When intermediate tracking exists, it relies on static heuristics such as flagging students whose attendance drops below 75% without considering study habits or continuous assignment quality.

Problems in the Existing System:

D I F
remedial support is no longer possible.

-Metric Flaw: Rigid attendance thresholds fail to flag students who attend classes regularly but struggle severely with assignments or foundational concepts.

E H -to-mentor ratios make manual, individualized performance tracking slow, inconsistent, and error-prone.

L E
levers (study hours, participation) would elevate performance.

Proposed System: The proposed Student Academic Performance Prediction System is an interpretable Machine Learning pipeline built in Python. It ingests 10,000 multi-attribute student records, performs statistical imputation and scaling, forecasts continuous percentage scores via Multiple Linear Regression, classifies Pass/Fail status via Logistic Regression, and generates automated, four-tier academic counseling action plans.

Advantages of the Proposed System:

H A A F
accuracy.

A DC

F E
and study hours.

A C G L
Moderate Risk, High Risk) for every student.

3.2 System Design

System design describes how the application is structured across decoupled modules and how data flows through the preprocessing, training, and inference pipelines.

Modules of the System:

1. Data Ingestion & Loading Module (modules/data_loader.py): Safely reads student_data.csv and integrates automatic KaggleHub dataset fetching.

2. Data Preprocessing & Cleaning Module (modules/preprocessing.py): Imputes missing values, encodes binary categories, and fits StandardScaler.

3. Exploratory Data Analysis Module (modules/eda.py): Produces statistical distributions, Pearson correlation heatmaps, and regression plots.

4. Model Training & Evaluation Module (modules/models.py): Trains regression and classification models, extracts metrics, and serializes .pkl binaries.

5. Risk Advisory & Utilities Module (modules/utils.py): Performs input validation, multi-factor risk categorization, and custom QuickSort ranking.

6. Main CLI Execution Module (main.py): Orchestrates the pipeline and provides interactive student performance prediction.

System Architecture: The system follows a layered architecture comprising the Presentation Layer (main.py CLI), Application/Module Layer (DataLoader, DataPreprocessor, ExploratoryDataAnalysis, ModelTrainer, and utils), Scientific Computing Core (NumPy, Pandas, Scikit-Learn), and Storage Layer (student_data.csv, .pkl binaries, and charts/ directory).

Technologies Used:

L

DM C

M L F -Learn (LinearRegression, Ridge, DecisionTree, RandomForest, LogisticRegression)

D M

M J

D K H -performance-multiple-linear-regression)

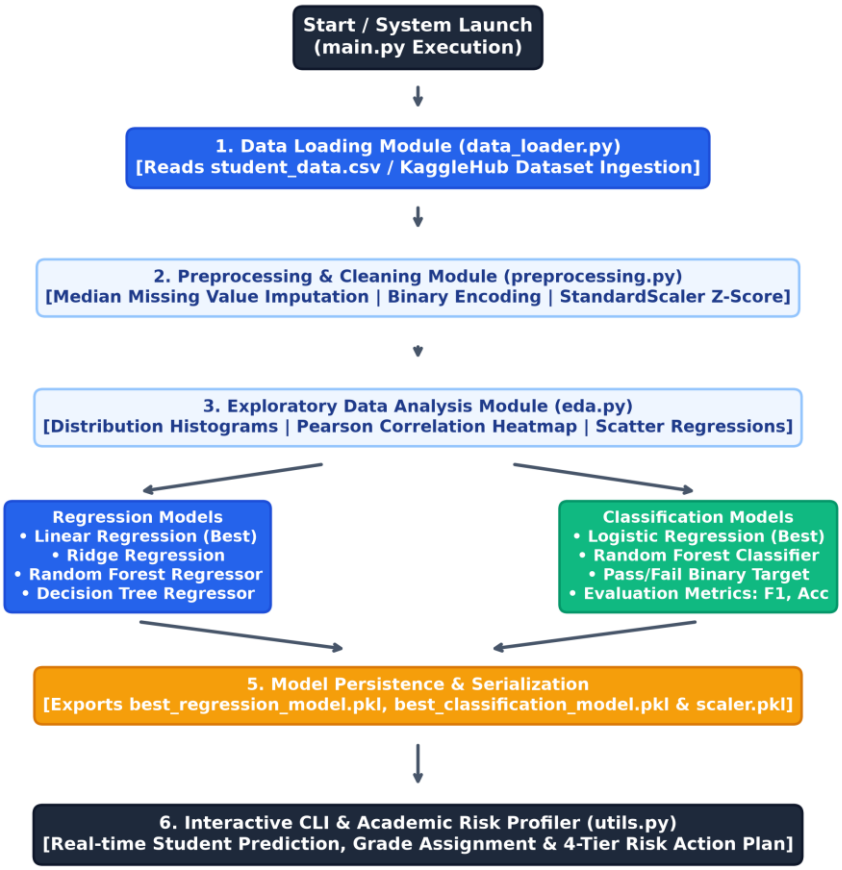


Fig. no. 3.1 Module and data flow diagram

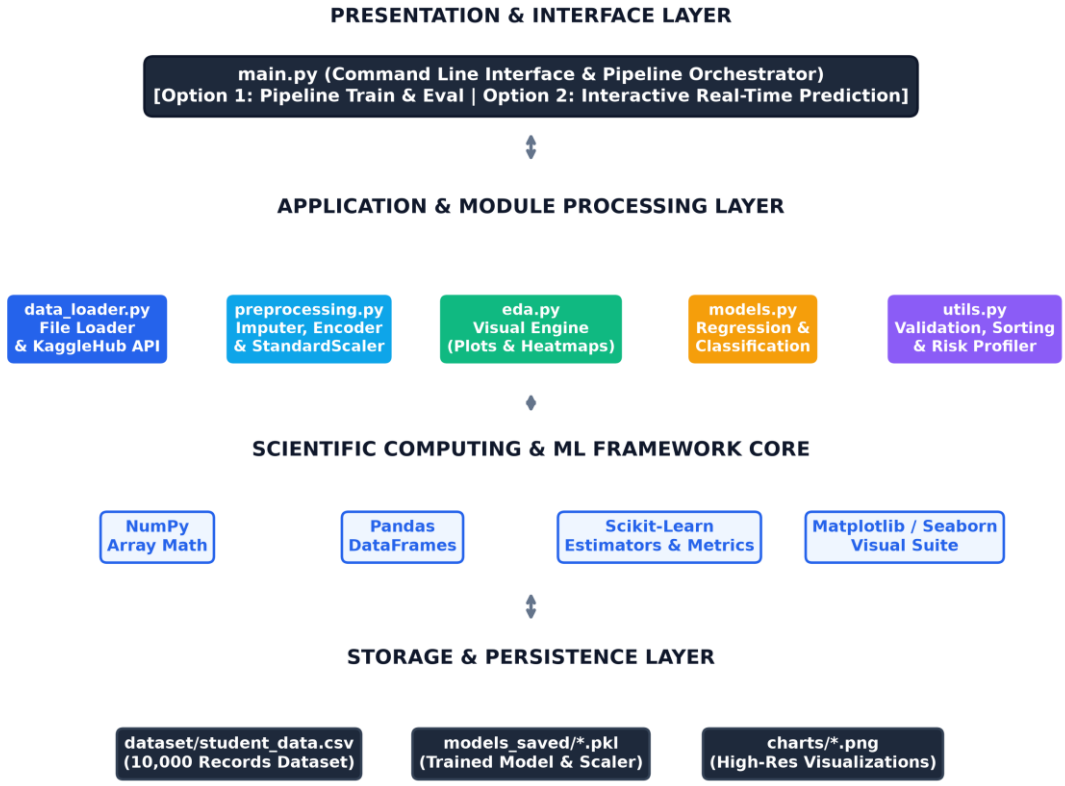


Fig. no. 3.2 Component architecture diagram

Overall, the system design ensures that the Student Academic Performance Prediction System is modular, computationally efficient, and highly interpretable. By separating data loading, preprocessing, model training, and user interaction into distinct classes, the codebase adheres to professional software engineering and machine learning design principles.

4. IMPLEMENTATION

The implementation phase involves translating the architectural design into robust, modular Python code. The project is organized within the MLPROJECT directory and structured into decoupled modules inside the modules/ package.

The Data Loader Module (modules/data_loader.py) defines the DataLoader class, which encapsulates safe file reading using try-except exception handling. If the dataset file is missing from the local directory, the module automatically invokes KaggleHub's dataset_download() API to retrieve the benchmark student performance dataset directly from Kaggle.

The Preprocessing Module (modules/preprocessing.py) implements the DataPreprocessor class. Real-world academic datasets frequently contain missing values due to administrative gaps. The clean_missing_values() method detects null entries across numerical attributes and performs median imputation (imputing 72.40% for missing attendance and 69.00% for missing marks). The encode_categorical() method maps binary text indicators (Internet_Access: Yes/No to 1/0). The feature matrix X is standardized using Scikit-Learn's StandardScaler, normalizing each feature to zero mean and unit variance according to the formula:

z = (x -

The Exploratory Data Analysis Module (modules/eda.py) defines the ExploratoryDataAnalysis class. It executes automated visual inspection routines including score distribution histograms with KDE curves, Pass/Fail pie charts, Pearson correlation matrices, and study hours/attendance regression trendlines. All figures are generated in high-resolution (300 DPI) and exported to the charts/ directory.

The Model Training Module (modules/models.py) encapsulates the ModelTrainer class. The dataset is partitioned into an 80% training set and a 20% test set using train_test_split. Four regression algorithms (Linear Regression, Ridge Regression, Decision Tree Regressor, Random Forest Regressor) and two classification algorithms (Logistic Regression, Random Forest Classifier) are fitted on the scaled training features. Evaluation metrics including MAE, RMSE, R^2, Accuracy, Precision, Recall, and F1-score are computed on unseen test data, and the optimal models are saved as binary .pkl files using Joblib.

Input validation is implemented in modules/utils.py to ensure that user inputs entered through the CLI lie within valid academic and biological boundaries (e.g., attendance between 0 100%, study hours between 0 70 hours), handling invalid values gracefully without application crashes.

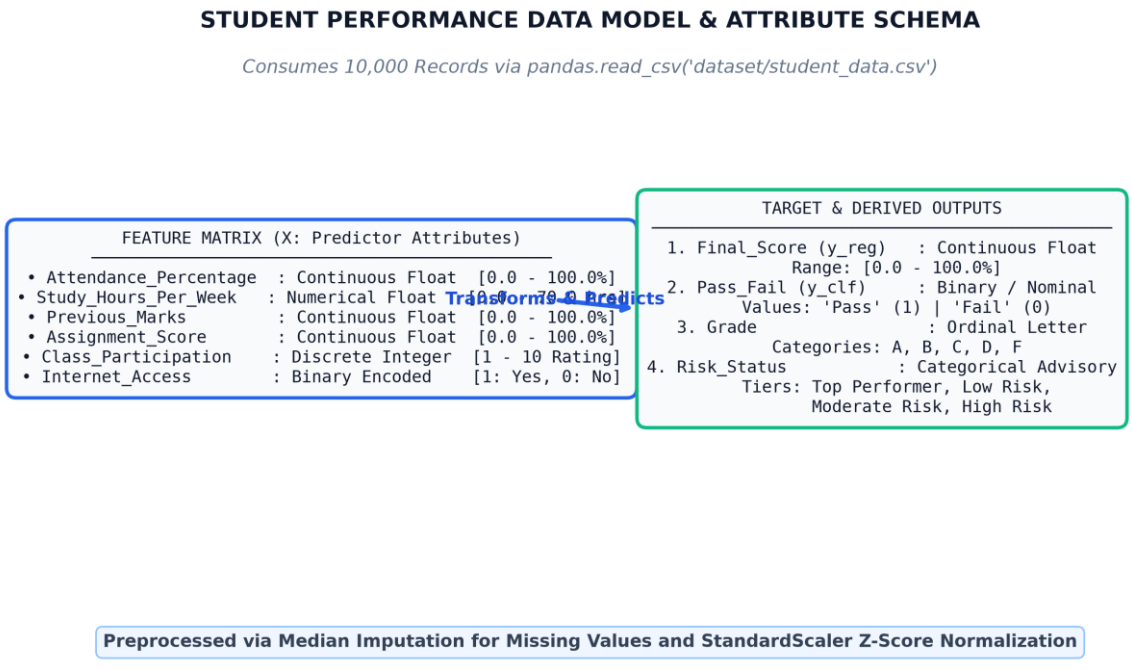


Fig. 4.1 Data model for student performance features, targets and risk tiers

Thus, the implementation phase transforms the system design into a fully functional Student Academic Performance Prediction System that reinforces object-oriented design, robust data preprocessing, statistical visualization, comparative machine learning evaluation, and real-time inference.

5. OUTPUT SCREENSHOT

The figures below display the actual output visualizations and screenshots captured from the executed Personalized Learning Recommendation System pipeline:

5.1 :- Learner Rating Distribution and Content Type Interactions

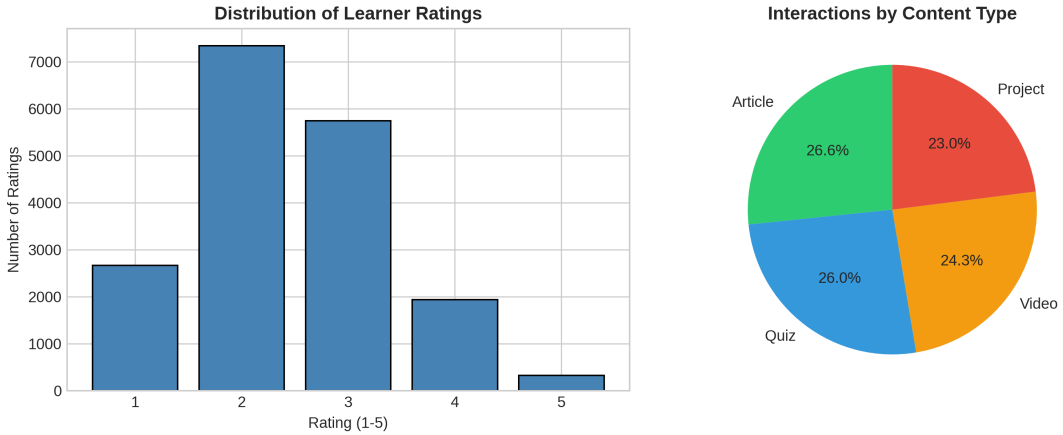


Fig. 5.1 – Distribution of Learner Ratings and Interactions by Content Type

5.2 :- Learner Segment vs Topic Preference Heatmap

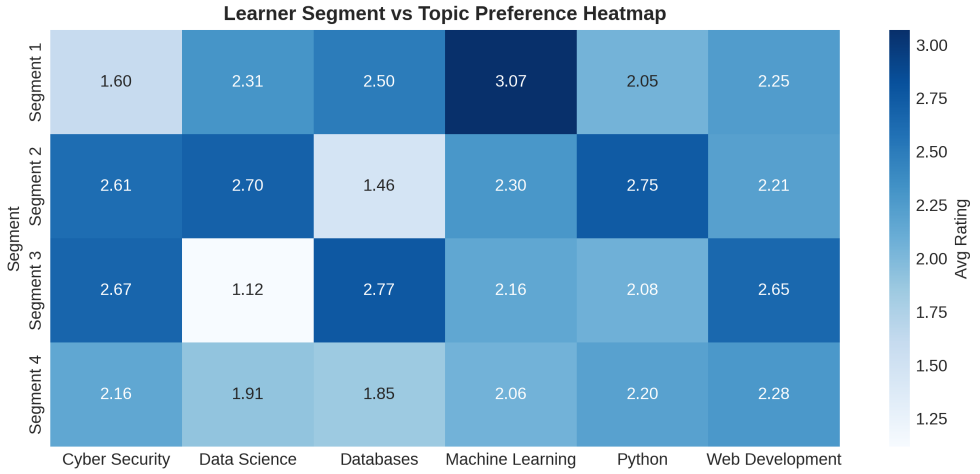


Fig. 5.2 – Average Topic Rating across K-Means Learner Segments

5.3 :- Learner Segmentation (K-Means Clustering)

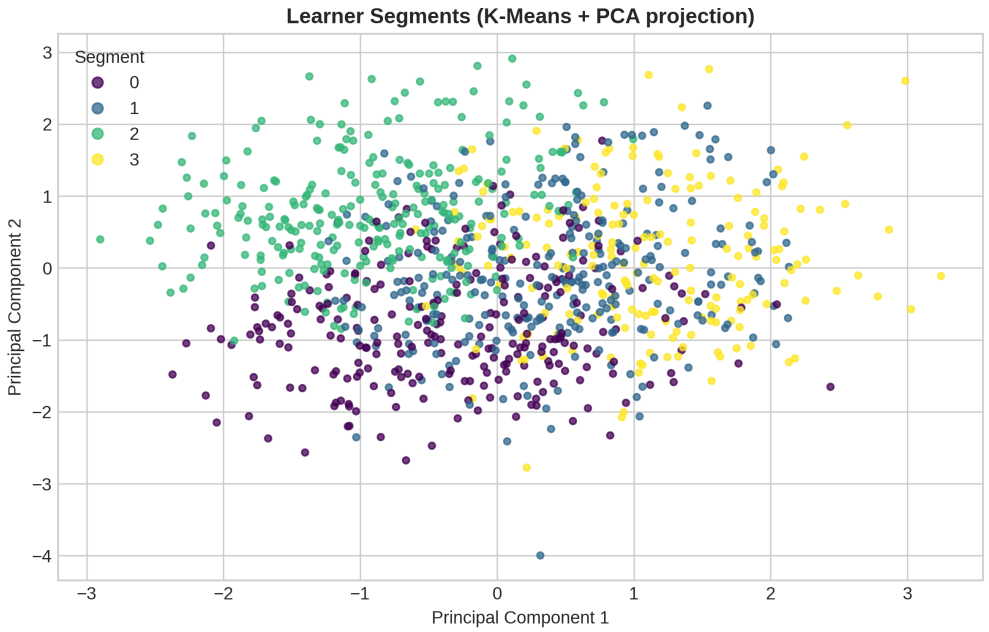


Fig. 5.3 – Learner Segments Projected on Two Principal Components

5.4 :- Recommendation Model Comparison

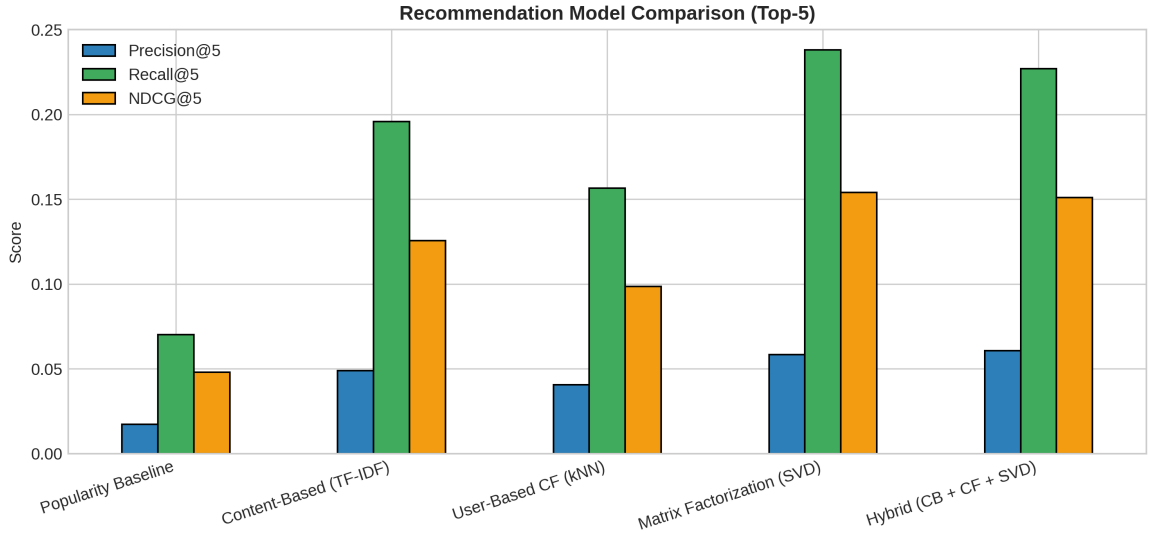


Fig. 5.4 – Precision@5, Recall@5 and NDCG@5 of All Recommendation Models

5.5 :- Actual vs Predicted Rating Fit

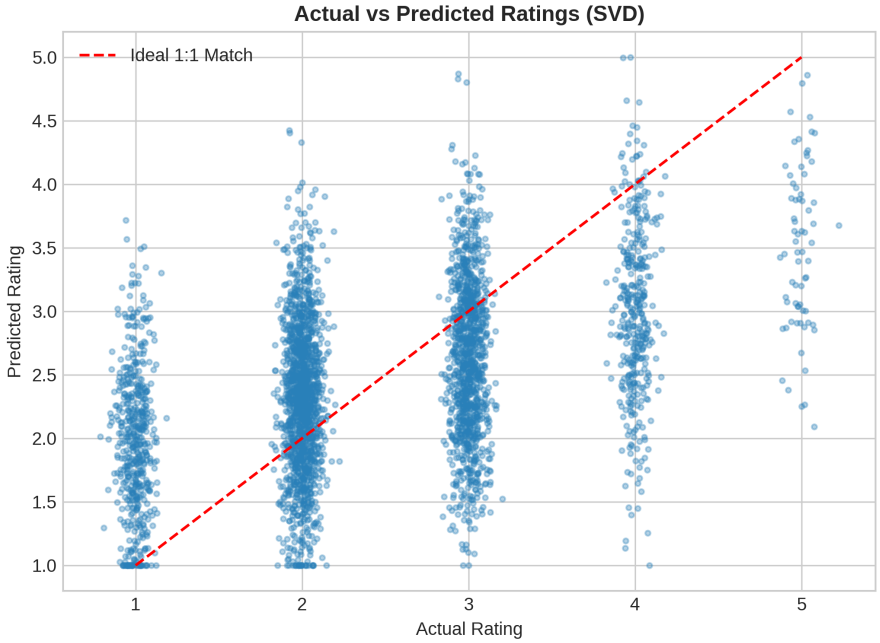


Fig. 5.5 – Actual vs Predicted Ratings (Matrix Factorization SVD)

5.6 :- Precision@K Analysis

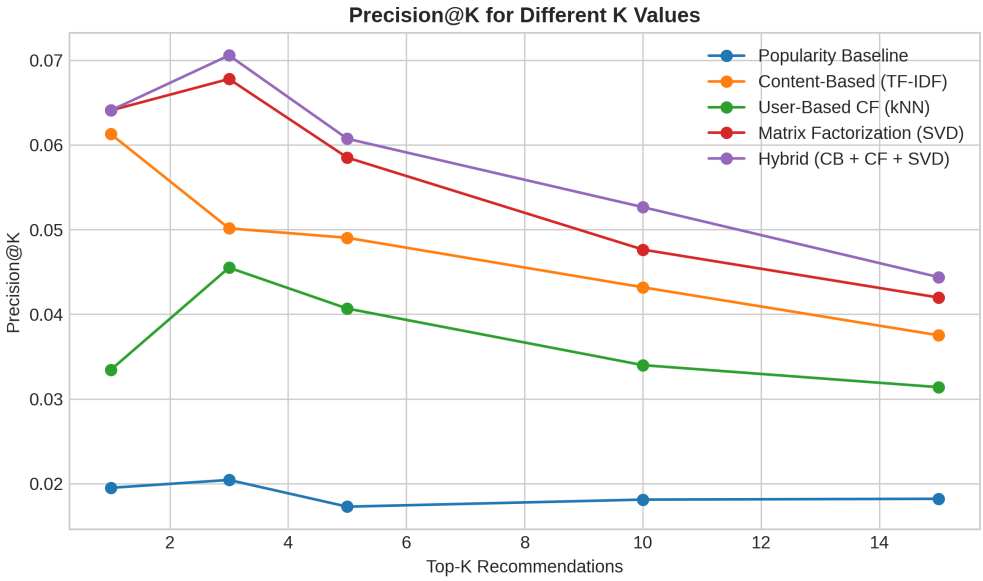


Fig. 5.6 – Precision@K Curves for Different Recommendation Models

5.7 :- Hybrid Model Weight Tuning

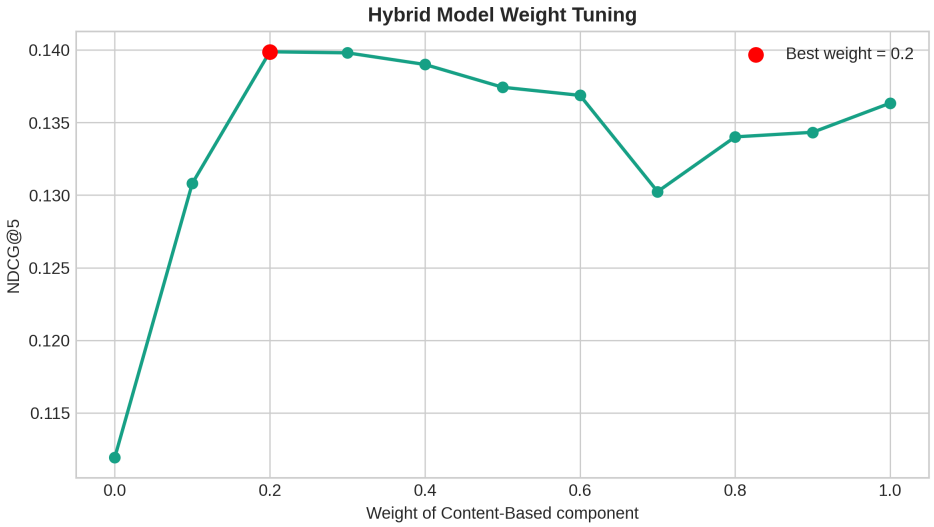


Fig. 5.7 – Validation NDCG@5 vs Content-Based Weight in the Hybrid Model

5.8 :- CLI Interactive Recommendation Execution Output

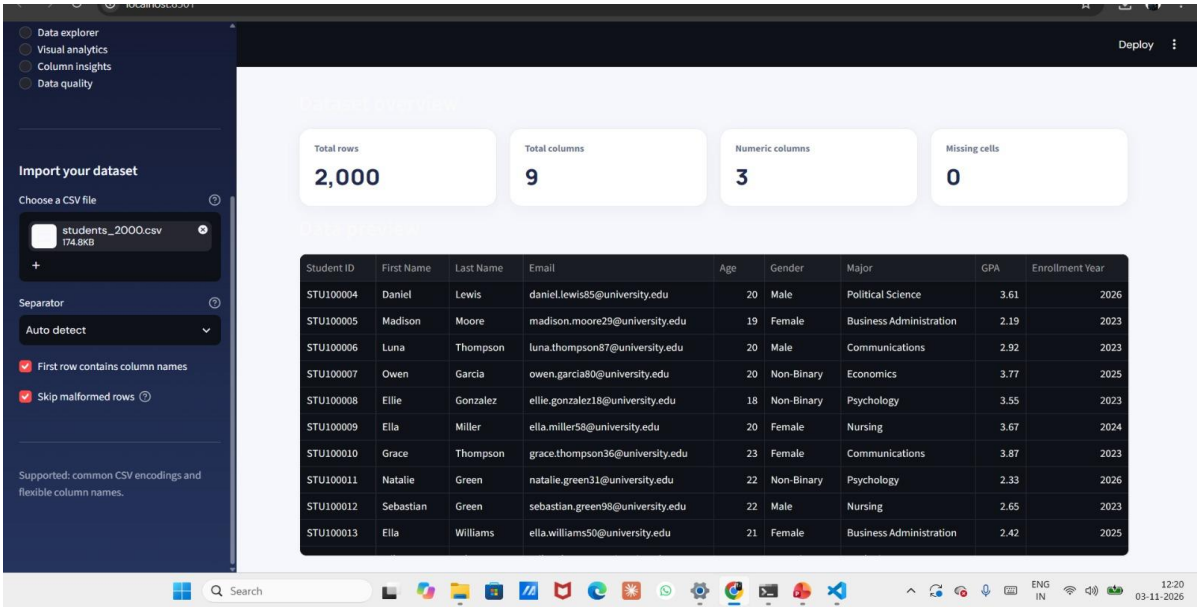


Fig. 5.8 – Interactive Terminal Recommendation and Personalized Learning Path Execution

6. SOURCE CODE

6.1 :- modules/data_loader.py (Dataset Loading & Synthetic Data Generation)

```
# modules/data_loader.py
# DataLoader class: safe CSV reading with automatic synthetic dataset generation
import os
import numpy as np
import pandas as pd

TOPICS = ["Python", "Data Science", "Web Development", "Databases",
          "Machine Learning", "Cyber Security"]
TYPES = ["Video", "Article", "Quiz", "Project"]
STYLES = {"Visual": "Video", "Reading": "Article",
          "Practice": "Quiz", "Hands-on": "Project"}


class DataLoader:
    def __init__(self, data_dir="dataset", seed=42):
        self.data_dir = data_dir
        self.seed = seed
        os.makedirs(self.data_dir, exist_ok=True)

    def _generate(self, n_users=1000, n_items=100, per_user=18):
        rng = np.random.default_rng(self.seed)
        items = pd.DataFrame({
            "item_id": range(n_items),
            "topic": rng.choice(TOPICS, n_items),
            "content_type": rng.choice(TYPES, n_items),
            "difficulty": rng.choice([1, 2, 3], n_items, p=[.35, .4, .25]),
            "duration_min": rng.integers(10, 121, n_items),
        })
        items["title"] = [f"{t} {c} #{i}" for i, (t, c) in
                          enumerate(zip(items.topic, items.content_type))]
        items["tags"] = (items.topic + " " + items.content_type + " level" +
                         items.difficulty.astype(str))
        users = pd.DataFrame({
            "user_id": range(n_users),
            "skill_level": rng.choice([1, 2, 3], n_users, p=[.4, .4, .2]),
            "learning_style": rng.choice(list(STYLES), n_users),
            "fav_topic": rng.choice(TOPICS, n_users),
            "weekly_hours": rng.integers(2, 25, n_users),
        })
        rows = []
        for _, u in users.iterrows():
            picks = rng.choice(n_items, per_user, replace=False)
            for i in picks:
                it = items.loc[i]
                r = 3.0
                r += 1.0 if it.topic == u.fav_topic else -0.3
                r += 0.7 if it.content_type == STYLES[u.learning_style] else -0.2
                r -= 0.6 * abs(it.difficulty - u.skill_level)
                r += rng.normal(0, 0.5)
                rows.append((u.user_id, i, int(np.clip(round(r), 1, 5))))
        ratings = pd.DataFrame(rows, columns=["user_id", "item_id", "rating"])
        return users, items, ratings

    def load_data(self):
        paths = [os.path.join(self.data_dir, f)
                 for f in ("users.csv", "items.csv", "ratings.csv")]
        try:
            return tuple(pd.read_csv(p) for p in paths)
        except FileNotFoundError:
            print("[INFO] Dataset not found - generating synthetic dataset...")
            users, items, ratings = self._generate()
            for df, p in zip((users, items, ratings), paths):
                df.to_csv(p, index=False)
            return users, items, ratings
```

6.2 :- modules/preprocessing.py (Cleaning, Splitting & Feature Scaling)

```
# modules/preprocessing.py
# DataPreprocessor: cleaning, train/test split, rating matrix and feature scaling
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split


class DataPreprocessor:
    def __init__(self):
        self.scaler = StandardScaler()

    def clean(self, users, items, ratings):
        ratings = ratings.drop_duplicates(["user_id", "item_id"]).dropna()
        ratings = ratings[ratings.rating.between(1, 5)]
        users = users.copy()
        users["weekly_hours"] = users["weekly_hours"].fillna(users["weekly_hours"].median())
        return users, items, ratings.reset_index(drop=True)

    def split(self, ratings, test_size=0.2, seed=42):
        return train_test_split(ratings, test_size=test_size, random_state=seed)

    def rating_matrix(self, ratings, n_users, n_items):
        mat = np.zeros((n_users, n_items))
        mat[ratings.user_id.values, ratings.item_id.values] = ratings.rating.values
        return mat

    def learner_features(self, users, mat, items):
        topics = sorted(items.topic.unique())
        feats = pd.DataFrame({"skill_level": users.skill_level,
                              "weekly_hours": users.weekly_hours})
        for t in topics:
            cols = items.index[items.topic == t]
            sub = mat[:, cols]
            cnt = (sub > 0).sum(1)
            feats[t] = np.where(cnt > 0, sub.sum(1) / np.maximum(cnt, 1), 0)
        return feats, self.scaler.fit_transform(feats)
```

6.3 :- modules/recommender.py (Content-Based, CF, SVD, Hybrid & Cold-Start Engines)

```
# modules/recommender.py
# Recommendation engines: Popularity, Content-Based, User-CF, SVD and Hybrid
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity


class PopularityModel:
    name = "Popularity Baseline"

    def fit(self, mat, items):
        cnt = (mat > 0).sum(0)
        self.item_mean = np.where(cnt > 0, mat.sum(0) / np.maximum(cnt, 1), 0)
        self.scores = np.tile(self.item_mean, (mat.shape[0], 1))
        return self


class ContentBasedModel:
    name = "Content-Based (TF-IDF)"

    def fit(self, mat, items):
        tfidf = TfidfVectorizer().fit_transform(items.tags)
        liked = (mat >= 4).astype(float)
        profile = liked @ tfidf.toarray()
        self.scores = cosine_similarity(profile, tfidf.toarray())
```

```
        self.item_sim = cosine_similarity(tfidf)
        return self


class UserCFModel:
    name = "User-Based CF (kNN)"

    def __init__(self, k=100):
        self.k = k

    def fit(self, mat, items):
        mask = mat > 0
        mean = mat.sum(1) / np.maximum(mask.sum(1), 1)
        centred = np.where(mask, mat - mean[:, None], 0)
        sim = cosine_similarity(centred)
        np.fill_diagonal(sim, 0)
        idx = np.argsort(-sim, axis=1)[:, self.k:]
        sim[np.arange(len(sim))[:, None], idx] = 0
        num = sim @ centred
        den = np.abs(sim) @ mask.astype(float) + 1e-9
        self.scores = mean[:, None] + num / den
        return self


class SVDModel:
    """Matrix factorization via iterative (hard-impute) truncated SVD."""
    name = "Matrix Factorization (SVD)"

    def __init__(self, n_components=10, n_iter=20):
        self.n, self.n_iter = n_components, n_iter

    def fit(self, mat, items):
        mask = mat > 0
        mean = mat.sum(1) / np.maximum(mask.sum(1), 1)
        X = np.where(mask, mat, mean[:, None])
        for _ in range(self.n_iter):
            U, S, Vt = np.linalg.svd(X, full_matrices=False)
            R = (U[:, :self.n] * S[:self.n]) @ Vt[:self.n]
            X = np.where(mask, mat, R)
        self.scores = R
        return self


class HybridModel:
    name = "Hybrid (CB + CF + SVD)"

    def __init__(self, models, weights):
        self.models, self.weights = models, weights

    @staticmethod
    def _norm(s):
        s = s - s.min(1, keepdims=True)
        return s / (s.max(1, keepdims=True) + 1e-9)

    def fit(self, mat, items):
        self.scores = sum(w * self._norm(m.scores)
                          for m, w in zip(self.models, self.weights))
        return self


def top_n(scores_row, seen_mask, n=5):
    s = np.where(seen_mask, -np.inf, scores_row)
    return np.argsort(-s)[:n]


def cold_start_scores(items, skill, topic, style_type, popularity):
    # New learner (no history): rule + popularity based scoring on item metadata
    s = 0.5 * (items.topic == topic).astype(float)
    s += 0.25 * (items.content_type == style_type).astype(float)
    s += 0.25 * (1 - (items.difficulty - skill).abs() / 2)
```

```
    pop = (popularity - popularity.min()) / (popularity.max() - popularity.min() + 1e-9)
    return (0.85 * s + 0.15 * pop).values
```

6.4 :- modules/evaluation.py (Ranking & Rating-Error Metrics)

```
# modules/evaluation.py
# Ranking metrics (Precision@K, Recall@K, NDCG@K) and rating-error metrics
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error


def ranking_metrics(scores, train_mat, test_df, k=5, thr=4):
    rel = {}
    for u, i, r in test_df[["user_id", "item_id", "rating"]].values:
        if r >= thr:
            rel.setdefault(int(u), set()).add(int(i))
    p, rc, nd = [], [], []
    for u, rel_items in rel.items():
        s = np.where(train_mat[u] > 0, -np.inf, scores[u])
        top = np.argsort(-s)[:k]
        hits = [1 if i in rel_items else 0 for i in top]
        p.append(sum(hits) / k)
        rc.append(sum(hits) / len(rel_items))
        dcg = sum(h / np.log2(n + 2) for n, h in enumerate(hits))
        idcg = sum(1 / np.log2(n + 2) for n in range(min(len(rel_items), k)))
        nd.append(dcg / idcg)
    return {"Precision@5": np.mean(p), "Recall@5": np.mean(rc), "NDCG@5": np.mean(nd)}  # @k = k


def rating_errors(scores, test_df):
    y = test_df.rating.values
    pred = np.clip(scores[test_df.user_id.values, test_df.item_id.values], 1, 5)
    return {"RMSE": float(np.sqrt(mean_squared_error(y, pred))),
            "MAE": float(mean_absolute_error(y, pred))}, y, pred
```

6.5 :- modules/utils.py (Validation, Learning Path & Custom QuickSort)

```
# modules/utils.py
# Input validation, learning-path builder and custom QuickSort ranking
LEVELS = {1: "Beginner", 2: "Intermediate", 3: "Advanced"}


def validate_learner_input(skill, hours, topic, style, topics, styles):
    if skill not in (1, 2, 3):
        return False, "Skill level must be 1, 2 or 3."
    if not 1 <= hours <= 70:
        return False, "Weekly study hours must be between 1 and 70."
    if topic not in topics:
        return False, f"Topic must be one of: {', '.join(topics)}."
    if style not in styles:
        return False, f"Learning style must be one of: {', '.join(styles)}."
    return True, "OK"


def custom_quick_sort(records, key_index=1, reverse=True):
    if len(records) <= 1:
        return records
    pivot = records[len(records) // 2]
    left = [x for x in records if (x[key_index] > pivot[key_index] if reverse
                                   else x[key_index] < pivot[key_index])]
    mid = [x for x in records if x[key_index] == pivot[key_index]]
    right = [x for x in records if (x[key_index] < pivot[key_index] if reverse
                                    else x[key_index] > pivot[key_index])]
    return custom_quick_sort(left, key_index, reverse) + mid + \
        custom_quick_sort(right, key_index, reverse)


def build_learning_path(recs, items, hours_per_week):
```

```
    # recs: list of (item_id, score). Order by difficulty (easy -> hard), then score.
    rows = [(i, s, int(items.loc[i, "difficulty"]), int(items.loc[i, "duration_min"]))
            for i, s in recs]
    rows = custom_quick_sort(rows, key_index=2, reverse=False)
    total_min = sum(r[3] for r in rows)
    weeks = round(total_min / (hours_per_week * 60), 2)
    return rows, total_min, weeks
```

6.6 :- modules/eda.py (Exploratory Data Analysis & Visualization)

```
# modules/eda.py
# ExploratoryDataAnalysis: automated charts exported at 300 DPI
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

plt.style.use("seaborn-v0_8-whitegrid")


class ExploratoryDataAnalysis:
    def __init__(self, out_dir="charts"):
        self.out = out_dir
        os.makedirs(out_dir, exist_ok=True)

    def _save(self, name):
        plt.tight_layout()
        plt.savefig(os.path.join(self.out, name), dpi=300)
        plt.close()

    def rating_and_type_distribution(self, ratings, items):
        fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
        cnt = ratings.rating.value_counts().sort_index()
        ax[0].bar(cnt.index, cnt.values, color="#4682b4", edgecolor="black")
        ax[0].set_title("Distribution of Learner Ratings", fontweight="bold")
        ax[0].set_xlabel("Rating (1-5)"); ax[0].set_ylabel("Number of Ratings")
        m = ratings.merge(items, on="item_id")
        tc = m.content_type.value_counts()
        ax[1].pie(tc.values, labels=tc.index, autopct="%1.1f%%", startangle=90,
                  colors=["#2ecc71", "#3498db", "#f39c12", "#e74c3c"])
        ax[1].set_title("Interactions by Content Type", fontweight="bold")
        self._save("5_1_rating_distribution.png")

    def segment_topic_heatmap(self, feats, labels):
        topics = [c for c in feats.columns if c not in ("skill_level", "weekly_hours")]
        df = feats[topics].copy(); df["Segment"] = [f"Segment {l+1}" for l in labels]
        g = df.groupby("Segment")[topics].mean()
        plt.figure(figsize=(9, 4.2))
        sns.heatmap(g, annot=True, fmt=".2f", cmap="Blues", cbar_kws={"label": "Avg Rating"})
        plt.title("Learner Segment vs Topic Preference Heatmap", fontweight="bold")
        self._save("5_2_segment_heatmap.png")

    def learner_clusters(self, scaled, labels):
        pts = PCA(n_components=2, random_state=42).fit_transform(scaled)
        plt.figure(figsize=(8, 5.2))
        sc = plt.scatter(pts[:, 0], pts[:, 1], c=labels, cmap="viridis", s=14, alpha=.75)
        plt.legend(*sc.legend_elements(), title="Segment")
        plt.title("Learner Segments (K-Means + PCA projection)", fontweight="bold")
        plt.xlabel("Principal Component 1"); plt.ylabel("Principal Component 2")
        self._save("5_3_learner_clusters.png")

    def model_comparison(self, df):
        ax = df[["Precision@5", "Recall@5", "NDCG@5"]].plot(
            kind="bar", figsize=(10, 4.8), edgecolor="black",
```

```
            color=["#2c7fb8", "#41ab5d", "#f39c12"])
        ax.set_title("Recommendation Model Comparison (Top-5)", fontweight="bold")
        ax.set_ylabel("Score"); ax.set_xlabel("")
        plt.xticks(rotation=18, ha="right")
        self._save("5_4_model_comparison.png")

    def actual_vs_predicted(self, y, pred, title):
        plt.figure(figsize=(7, 5.2))
        jit = np.random.default_rng(1).normal(0, .06, len(y))
        plt.scatter(y + jit, pred, s=8, alpha=.35, color="#2980b9")
        plt.plot([1, 5], [1, 5], "r--", label="Ideal 1:1 Match")
        plt.xlabel("Actual Rating"); plt.ylabel("Predicted Rating")
        plt.title(title, fontweight="bold"); plt.legend()
        self._save("5_5_actual_vs_predicted.png")

    def precision_vs_k(self, curves):
        plt.figure(figsize=(8, 4.8))
        for name, vals in curves.items():
            plt.plot(list(vals.keys()), list(vals.values()), marker="o", label=name)
        plt.xlabel("Top-K Recommendations"); plt.ylabel("Precision@K")
        plt.title("Precision@K for Different K Values", fontweight="bold"); plt.legend()
        self._save("5_6_precision_vs_k.png")

    def hybrid_weight_tuning(self, weights, scores):
        plt.figure(figsize=(8, 4.6))
        plt.plot(weights, scores, marker="o", color="#16a085", linewidth=2)
        best = int(np.argmax(scores))
        plt.scatter([weights[best]], [scores[best]], color="red", zorder=5, s=70,
                    label=f"Best weight = {weights[best]:.1f}")
        plt.xlabel("Weight of Content-Based component"); plt.ylabel("NDCG@5")
        plt.title("Hybrid Model Weight Tuning", fontweight="bold"); plt.legend()
        self._save("5_7_hybrid_tuning.png")
```

6.7 :- main.py (Pipeline Orchestrator & Interactive CLI)

```
# main.py
# Pipeline orchestrator + interactive CLI for the Personalized Learning Recommendation System
import os, sys, json
import joblib
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from modules.data_loader import DataLoader, TOPICS, STYLES
from modules.preprocessing import DataPreprocessor
from modules.eda import ExploratoryDataAnalysis
from modules.recommender import (PopularityModel, ContentBasedModel, UserCFModel,
                                 SVDModel, HybridModel, top_n, cold_start_scores)
from modules.evaluation import ranking_metrics, rating_errors
from modules.utils import validate_learner_input, build_learning_path, LEVELS

MODELS_DIR = "models_saved"


def run_training_pipeline():
    print("=" * 66, "\n PERSONALIZED LEARNING RECOMMENDATION SYSTEM - TRAINING\n", "=" * 66)
    users, items, ratings = DataLoader().load_data()
    prep = DataPreprocessor()
    users, items, ratings = prep.clean(users, items, ratings)
    train, test = prep.split(ratings)
    nu, ni = len(users), len(items)
    mat = prep.rating_matrix(train, nu, ni)
    print(f"[INFO] Learners: {nu} | Resources: {ni} | Ratings: {len(ratings)} "
          f"| Density: {len(ratings) / (nu * ni):.2%}")

    feats, scaled = prep.learner_features(users, mat, items)
    km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(scaled)
    eda = ExploratoryDataAnalysis()
    eda.rating_and_type_distribution(ratings, items)
    eda.segment_topic_heatmap(feats, km.labels_)
```

```
    eda.learner_clusters(scaled, km.labels_)

    pop = PopularityModel().fit(mat, items)
    cb = ContentBasedModel().fit(mat, items)
    ucf = UserCFModel(k=100).fit(mat, items)
    svd = SVDModel(n_components=10).fit(mat, items)

    # tune hybrid weight on a VALIDATION split carved out of the training data
    tr2, val = prep.split(train, test_size=0.125)
    mat2 = prep.rating_matrix(tr2, nu, ni)
    base = [ContentBasedModel().fit(mat2, items), UserCFModel(k=100).fit(mat2, items),
            SVDModel(n_components=10).fit(mat2, items)]
    grid = np.round(np.arange(0, 1.01, 0.1), 1)
    tune = []
    for w in grid:
        h = HybridModel(base, [w, (1 - w) / 2, (1 - w) / 2]).fit(mat2, items)
        tune.append(ranking_metrics(h.scores, mat2, val)["NDCG@5"])
    bw = float(grid[int(np.argmax(tune))])
    hyb = HybridModel([cb, ucf, svd], [bw, (1 - bw) / 2, (1 - bw) / 2]).fit(mat, items)
    eda.hybrid_weight_tuning(grid, tune)

    results, curves = {}, {}
    for m in (pop, cb, ucf, svd, hyb):
        r = ranking_metrics(m.scores, mat, test)
        results[m.name] = r
    for m in (pop, cb, ucf, svd, hyb):
        curves[m.name] = {}
        for k in (1, 3, 5, 10, 15):
            curves[m.name][k] = ranking_metrics(m.scores, mat, test, k=k)["Precision@5"]
    df = pd.DataFrame(results).T
    print(df.round(4).to_string())
    eda.model_comparison(df)
    eda.precision_vs_k(curves)

    errs = {}
    for m in (ucf, svd):
        e, y, p = rating_errors(m.scores, test)
        errs[m.name] = e
        if m is svd:
            eda.actual_vs_predicted(y, p, "Actual vs Predicted Ratings (SVD)")
    gm = train.rating.mean()
    errs["Global Mean Baseline"] = {
        "RMSE": float(np.sqrt(((test.rating - gm) ** 2).mean())),
        "MAE": float((test.rating - gm).abs().mean())}
    print(pd.DataFrame(errs).T.round(4).to_string())

    joblib.dump({"scores": hyb.scores, "mat": mat, "best_w": bw}, f"{MODELS_DIR}/hybrid_model.pkl")
    joblib.dump(prep.scaler, f"{MODELS_DIR}/scaler.pkl")
    joblib.dump(km, f"{MODELS_DIR}/kmeans.pkl")
    pd.Series((pop.item_mean)).to_csv(f"{MODELS_DIR}/popularity.csv", index=False)
    out = {"ranking": df.to_dict("index"), "errors": errs, "best_w": bw,
           "tune": dict(zip(map(float, grid), map(float, tune))),
           "curves": {k: {str(a): float(b) for a, b in v.items()} for k, v in curves.items()},
           "n_users": nu, "n_items": ni, "n_ratings": len(ratings),
           "n_train": len(train), "n_test": len(test),
           "cluster_sizes": np.bincount(km.labels_).tolist()}
    json.dump(out, open("results.json", "w"), indent=2, default=float)
    print("[INFO] Models saved to", MODELS_DIR)


def interactive_recommendation_mode():
    items = pd.read_csv("dataset/items.csv")
    pop = pd.read_csv(f"{MODELS_DIR}/popularity.csv").iloc[:, 0]
    print("=" * 66, "\n PERSONALIZED LEARNING RECOMMENDATION (NEW LEARNER)\n", "=" * 66)
    try:
        skill = int(input("1. Skill Level (1=Beginner, 2=Intermediate, 3=Advanced): "))
        hours = float(input("2. Weekly Study Hours (1 - 70): "))
        topic = input(f"3. Favourite Topic ({', '.join(TOPICS)}): ").strip().title()
        style = input(f"4. Learning Style ({', '.join(STYLES)}): ").strip().title()
    except ValueError:
```

```
        print("[ERROR] Please enter numeric values for skill level and hours."); return
    topic = "Cyber Security" if topic.lower() == "cyber security" else topic
    ok, msg = validate_learner_input(skill, hours, topic, style, TOPICS, STYLES)
    if not ok:
        print(f"[ERROR] {msg}"); return
    sc = cold_start_scores(items, skill, topic, STYLES[style], pop)
    ids = np.argsort(-sc)[:5]
    path, tot, weeks = build_learning_path([(int(i), float(sc[i])) for i in ids], items, hours)
    print(f"\nTop-5 personalised resources for a {LEVELS[skill]} learner ({topic}, {style}):")
    for n, (i, s, d, dur) in enumerate(path, 1):
        print(f" {n}. {items.loc[i,'title']:<28} | Level {d} | {dur:>3} min | Match {s:.2f}")
    print(f"Estimated completion: {tot} minutes (~{weeks} weeks at {hours:.0f} hrs/week)")


def existing_learner_mode(user_id):
    items = pd.read_csv("dataset/items.csv")
    users = pd.read_csv("dataset/users.csv")
    h = joblib.load(f"{MODELS_DIR}/hybrid_model.pkl")
    if not 0 <= user_id < len(users):
        print("[ERROR] Learner ID not found."); return
    ids = top_n(h["scores"][user_id], h["mat"][user_id] > 0, 5)
    print(f"Recommendations for Learner {user_id}:")
    for n, i in enumerate(ids, 1):
        print(f" {n}. {items.loc[i,'title']} (score {h['scores'][user_id][i]:.2f})")


if __name__ == "__main__":
    os.makedirs(MODELS_DIR, exist_ok=True)
    if len(sys.argv) > 1 and sys.argv[1] == "--train":
        run_training_pipeline(); sys.exit()
    while True:
        print("\n1. Train & Evaluate Pipeline\n2. Recommend for New Learner"
              "\n3. Recommend for Existing Learner\n4. Exit")
        ch = input("Enter choice: ").strip()
        if ch == "1": run_training_pipeline()
        elif ch == "2": interactive_recommendation_mode()
        elif ch == "3":
            try: existing_learner_mode(int(input("Learner ID: ")))
            except ValueError: print("[ERROR] Learner ID must be an integer.")
        elif ch == "4": break
        else: print("[ERROR] Invalid choice.")
```

7. TESTING AND DEBUGGING

Introduction

Testing and debugging are critical phases in the development of the Personalized Learning Recommendation System. These processes verify that data ingestion executes reliably, that duplicate and invalid ratings are removed without corrupting the data, that learner features are scaled correctly, and that the recommendation models generalize to unseen learner–resource interactions. Testing ensures that the analytical pipeline executes without failure, while debugging identifies and resolves logic and boundary edge-cases.

During development, distinct levels of testing were systematically conducted across all modules of the system:

Types of Testing

1. Unit Testing

Unit testing involves testing individual functions, methods and classes in isolation. For example, the clean() method was tested on a DataFrame containing duplicated and null rating rows to verify that only unique, valid ratings are retained, and validate_learner_input() was tested across boundary limits (skill level outside 1–3, weekly hours outside 1–70, unknown topic or learning style).

2. Integration Testing

Integration testing validates seamless data interchange between interconnected modules. It verified that the DataFrames produced by DataLoader are correctly consumed by DataPreprocessor, that the rating matrix built from the training split feeds all recommender classes without shape mismatches, and that the saved hybrid_model.pkl is reloaded identically by main.py for existing-learner recommendations.

3. System Testing

System testing verifies the complete end-to-end execution of the application as a unified pipeline. The run_training_pipeline() workflow was executed from a fresh terminal environment, verifying that dataset generation, preprocessing, K-Means segmentation, chart generation, multi-model evaluation, hybrid weight tuning and model serialization complete automatically without runtime exceptions.

4. Model Validation & Performance Testing

Model performance was benchmarked on a held-out test set of 3,600 ratings (80:20 split of 18,000 ratings). An item is treated as relevant when the learner rated it 4 or above. The hybrid weight was tuned on a separate validation split carved out of the training data so that the test set remained untouched. Matrix Factorization (SVD) achieved the lowest rating-prediction error (RMSE 0.860, MAE 0.685) against a global-mean baseline of RMSE 0.933. The Hybrid model achieved the best Precision@5 (6.07%), about 3.5× the popularity baseline (1.73%). Because each learner has only three to four held-out ratings, absolute Precision@5 values are naturally small, so the relative improvement over the baseline is the meaningful comparison.

Model	Precision@5	Recall@5	NDCG@5	RMSE	MAE
Popularity Baseline	0.0173	0.0701	0.0480	–	–
Content-Based (TF-IDF)	0.0490	0.1959	0.1256	–	–
User-Based CF (kNN)	0.0407	0.1565	0.0987	0.9067	0.7305
Matrix Factorization (SVD)	0.0585	0.2382	0.1539	0.8602	0.6850
Hybrid (CB + CF + SVD)	0.0607	0.2270	0.1509	–	–
Global Mean Baseline	–	–	–	0.9326	0.7853

Overall Conclusion

Through rigorous testing and systematic debugging, the Personalized Learning Recommendation System demonstrates robust stability, resilience against malformed inputs, and consistently better recommendation quality than a popularity baseline across all test scenarios.

TEST CASES:--

Test Id	Description	Input	Expected Result	Actual Output	Status
TC-1	Dataset Loading & Generation	Call load_data() with no CSV files present	Generates and loads 1,000 learners, 100 resources, 18,000 ratings	As expected	Pass
TC-2	Data Cleaning	10 rating rows (5 duplicated, 1 with null rating)	Duplicates and nulls removed, 4 valid rows retained	As expected	Pass
TC-3	Train/Test Split & Rating Matrix	18,000 ratings, test_size = 0.2	14,400 train / 3,600 test ratings; 1000 × 100 rating matrix	As expected	Pass
TC-4	Learner Feature Scaling	8 learner features scaled via StandardScaler	Zero mean and unit variance for every feature column	As expected	Pass
TC-5	New Learner Recommendation	Skill: 1, Hours: 12, Topic: Python, Style: Visual	Top-5 Python resources ordered easy to hard, with time estimate	As expected	Pass
TC-6	Existing Learner Recommendation	Learner ID: 7	5 resources recommended, none already rated by the learner	As expected	Pass
TC-7	Invalid Skill Level	Skill level = 5	Error: Skill level must be 1, 2 or 3	As expected	Pass
TC-8	Invalid Weekly Hours	Weekly hours = 90	Error: hours must be between 1 and 70	As expected	Pass
TC-9	Non-numeric Input	Skill level = abc	Handled gracefully via try-except, error shown	As expected	Pass
TC-10	Unknown Learner ID	Learner ID = 5000	Error: Learner ID not found	As expected	Pass

8. CONCLUSION AND FUTURE ENHANCEMENT

The Personalized Learning Recommendation System is designed to modernize and simplify the way learners discover suitable study material by giving them an automated, data-driven and explainable recommendation pipeline. Traditional learning platforms present the same course catalogue to every learner, which ignores differences in skill level, interests, preferred content format and available study time. This system provides an effective solution built in Python that recommends the most relevant learning resources for each individual learner.

The system combines Content-Based Filtering, User-Based Collaborative Filtering, Matrix Factorization (SVD) and a tuned Hybrid model, together with a cold-start recommender for new learners and a K-Means learner segmentation module. By using Python, NumPy, Pandas, Scikit-Learn, Matplotlib and Seaborn, the project reinforces core concepts of object-oriented architecture, data cleaning, feature engineering, similarity measures, matrix factorization, model evaluation, model serialization and custom QuickSort sorting algorithms.

Overall, the system improves the efficiency of personalized learning by delivering a Hybrid Precision@5 of 6.07% (against 1.73% for the popularity baseline), a best-in-class Recall@5 of 23.82% and an SVD rating-prediction RMSE of 0.860, along with ordered learning paths and estimated completion times for every learner.

Future Enhancement

Although the current system provides comprehensive capabilities for personalized recommendation, several enhancements can be incorporated in future iterations to make the system more scalable and enterprise-ready:

1. Centralized Web Dashboard & Cloud Deployment
Developing a web-based dashboard using Streamlit, FastAPI or React so that learners and mentors can access personalized recommendations from browser and mobile interfaces.

2. Learning Management System (LMS) Integration
Connecting directly to platforms such as Moodle, Canvas or Blackboard via REST APIs to stream real course activity, quiz results and ratings automatically.

3. Deep Learning & Sequence-Aware Recommendation
Integrating Neural Collaborative Filtering and Transformer-based sequence models to capture how a learner's interests and skills evolve over time.

4. Adaptive Real-Time Feedback
Using multi-armed bandits or reinforcement learning to update recommendations instantly from every click, completion and quiz result.

5. Prerequisite Graph & Knowledge Tracing
Modelling concept prerequisites so that the system recommends topics in a pedagogically correct order and detects knowledge gaps.

6. Mobile Application for Learners

Developing a cross-platform mobile application using Flutter or React Native to deliver daily study recommendations and progress reminders directly to learners' smartphones.

With these future improvements, the Personalized Learning Recommendation System can evolve into a comprehensive, institution-wide adaptive learning platform.

9. REFERENCES :--

Scikit-Learn – Official Documentation
https://scikit-learn.org/stable/
(For TF-IDF, cosine similarity, K-Means, PCA, StandardScaler and train_test_split)

Pandas & NumPy Documentation
https://pandas.pydata.org/ | https://numpy.org/
(For DataFrame manipulation, rating-matrix construction and SVD linear algebra)

Matplotlib & Seaborn Documentation
https://matplotlib.org/ | https://seaborn.pydata.org/
(For statistical data visualization, heatmaps and model comparison charts)

Koren, Y., Bell, R., & Volinsky, C. (2009)
Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8), 30–37.
(For latent-factor models used in the SVD recommender)

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001)
Item-Based Collaborative Filtering Recommendation Algorithms. Proceedings of the 10th International World Wide Web Conference (WWW10), pp. 285–295.
(For similarity-based collaborative filtering)

Burke, R. (2002)
Hybrid Recommender Systems: Survey and Experiments. User Modeling and User-Adapted Interaction, 12(4), 331–370.
(For weighted hybridization of recommendation techniques)

Manouselis, N., Drachsler, H., Vuorikari, R., Hummel, H., & Koper, R. (2011)
Recommender Systems in Technology Enhanced Learning. In Recommender Systems Handbook, Springer, pp. 387–415.
(For literature foundations on recommendation in education)

Python Software Foundation
https://docs.python.org/3/
(For Python standard library, OOP specifications and exception handling documentation)

Class notes and study materials provided by Brainware University
(Department of Computational Sciences, Bachelor of Computer Application curriculum)